The Reality Code Engine: One Form, Many Fits, and the Low-Entropy Path to Autonomous Coding

David Charlot, Open Interface Engineering

Research paper · OpenIE research.openie.dev
https://research.openie.dev/papers/rce/ · PDF https://research.openie.dev/pdfs/rce.pdf

Measurement. Research paper, not a journal final. Handler figures are est_j. Track B figures are reported_j from a RAPL package counter less a stated idle baseline. measured_j is reserved for a wall meter and is empty. No FPGA board result in this paper. No fabricated citations.

There are two paths to autonomous computer automation. One spends high energy on high entropy. The other converges on a low-entropy floor. A program is a database functional state machine: one Form, many fits. The Reality Code Engine (RCE) retrieves, adapts and proves fits instead of generating them. Correctness is the gate. Joules are the ranking. On a frozen 20-task repair corpus it resolves 20 of 20 with 0 model calls, at 84.2 reported joules per solved task.

One Form. Many fits. The gate is correctness, and the ranking is joules.

The economic reading of this paper is in its companion, The One-Path Error: The Economics of Autonomous Coding. This paper holds the engineering and the theory only.

---

1. Thesis

There are two paths to autonomous computer automation. They are told apart by two quantities: the energy spent per correct result, and the entropy the system must resolve at run time.

The first path generates. A large language model (LLM), a statistical text model, writes each change from the prompt. Every task pays for generation again, and every output is a fresh sample from a high-entropy distribution.

The second path converges. People have already coded most of what will ever be coded. A program is a database functional state machine: one Form, many fits. Human intent is high entropy. The machine floor is low entropy. Code converges by design, so most of the work is to retrieve a fit that exists, adapt it, and prove it. The Reality Code Engine (RCE) is built on the second path.

Both paths face the same gate. The gate is correctness: a task's completeness predicate $C(z)$, here the task's tests, a proof or an agreement trace. Among runs that pass the gate, systems are ranked by joules to completion, J^*. Correctness is a constraint, not a score.

Mixture of Limits is the philosophy: there is a minimal amount of anything to accomplish a task, and any more is waste (Metered Commit Gates, Part II). RCE is a Mixture of Limits instance. Its gears run cheapest first, and the first gear that passes the gate closes the task.

flowG is a stand-alone language project. RCE is its academic first-pass demonstration. The two stay separate projects.

Energy figures are labeled by source. `est_j` is an analytical estimate. `reported_j` is an operating-system or on-chip counter. `measured_j` is reserved for a wall meter and is empty in every result here.

2. Two paths to autonomy

One sentence in. One floor out.

How human intent becomes something a computer can run, and why it loses meaning on the way.

1. HUMAN INTENT IS HIGH ENTROPY

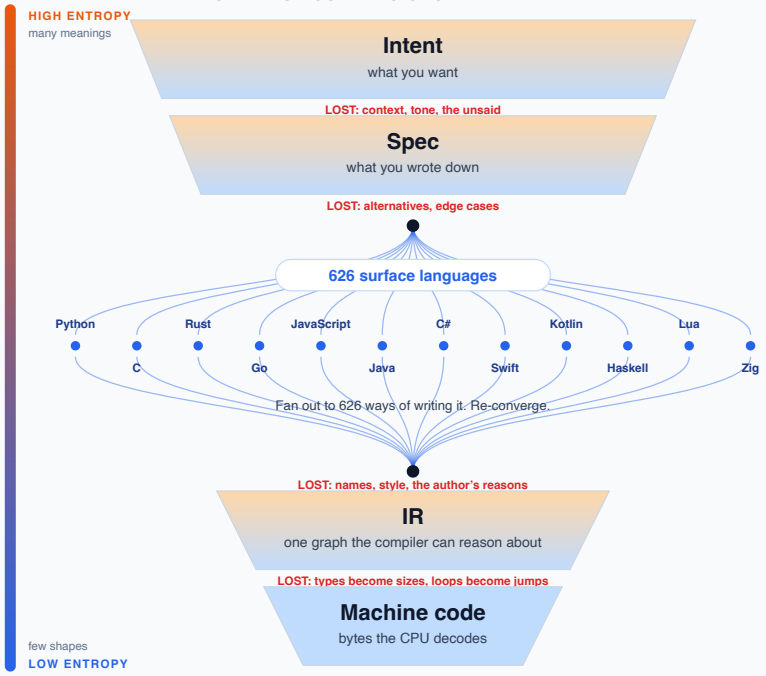
“Make the checkout faster.”

- Cut page load time?
- Fewer steps for the buyer?
- Faster card approval?
- Cache the cart?
- Faster for whom: phone or desktop?

Many meanings live in one sentence.
Context is implied, never written.
The listener fills the gap.
A computer cannot.

Shannon measured English at 0.6 to 1.3 bits per letter.
The letters are predictable. The meaning is not.

2. THE FUNNEL OF CONVERSIONS



3. It all gets to assembly.

`mov` `add` `cmp` `jmp` `load` `store` `call` `ret`

5. WHERE THE LOSS HAPPENS

Same line. Different answer.

Operators agree on easy inputs.
They split at the edges.

expression	Python	C
`-3 % 2`	`1`	`-1`
`-7 / 2`	`-3.5`	`-3`
`MAX + 1`	`grows`	`undefined`

Every boundary is a guess about what you meant.

Most guesses are right. Bugs live in the rest.

4. WHY ALL CODE STARTS TO LOOK THE SAME

The destination has room for only a few shapes.

Every program ends as loads, stores, math and jumps.
So code bends toward those shapes.

Convergence is by design. Not by laziness.

70% **of code on GitHub is clones.**
428M files, 85M unique.

DéjàVu, Lopes et al., OOPSLA 2017

6. THE FIX: RCE

Stop guessing at every boundary.

1. **One canonical form**
   Write the meaning once.
   One structure holds the intent.
2. **Every surface is a projection**
   Python, Rust, WASM, XState.
   Each one proven to match the form.
3. **Cheapest correct form wins**
   Equivalent forms, ranked by joules.
   Pick the one that costs least.

Sources: Shannon, “Prediction and Entropy of Printed English,” Bell System Technical Journal, 1951. Lopes et al., “DéjàVu: A Map of Code Duplicates on GitHub,” OOPSLA 2017.
626 languages: David Charlot’s lowering-path survey. Python and C results are language spec behavior. C signed overflow is undefined behavior.

Figure 1. Intent entropy and the machine floor. A PNG copy is provided. Sources: Shannon (1951); Lopes et al. (2017); the 626-language count is David Charlot's lowering-path survey. The Python and C results are language specification behavior; C signed overflow is undefined behavior.

Intent is high entropy. "Make the checkout faster" carries many meanings: shorter page load, fewer steps, faster card approval, a cached cart, or faster on one device class. Context is implied and never written. Shannon measured printed English at about 0.6 to 1.3 bits per letter [1]. The letters are predictable. The meaning is not.

Every conversion loses information. Intent becomes a specification, and context is lost. The specification becomes one of hundreds of surface languages, and alternatives and edge cases are lost. The source becomes an intermediate representation (IR), and names, style and the author's reasons are lost. The IR becomes machine code, and types become sizes and loops become jumps.

The floor is low entropy. Every program ends as loads, stores, arithmetic and jumps. The destination has room for only a few shapes, so code bends toward them. Convergence is by design.

Boundaries are where meaning diverges. The same line gives different answers at the edges. In Python, `-3 % 2` is 1 and `-7 / 2` is -3.5. In C, `-3 % 2` is -1, `-7 / 2` is -3, and signed overflow is undefined. A system that regenerates code at every boundary guesses at every boundary.

The two paths, stated as quantities.

Path	What runs per task	Entropy resolved at run time	Energy per correct result
Generate	A model samples a new change	The full distribution of the prompt	Paid again on every task
Converge (RCE)	Lookup, rule, solver; a model only last	Only the residual that no stored fit covers	Paid once per new case; then the handler floor

RCE writes the meaning once, in one canonical Form. Every surface language is a projection of that Form, checked to match it. Among equivalent forms, the cheapest correct one runs.

---

3. The Form

3.1 The strongest formal statement of "same form"

Gurevich's Abstract State Machine (ASM) theorem is the strongest formal statement that every algorithm has one shape [2]. An abstract state machine is a machine whose states are first-order structures and whose step is a finite set of guarded updates. Gurevich states three postulates. Sequential Time: an algorithm has states, initial states and a one-step map τ. Abstract State: states are first-order structures over one fixed vocabulary, closed under isomorphism. Bounded Exploration: a finite set of terms decides each step. His Main Theorem (Theorem 6.13) reads:

“For every sequential algorithm A, there exists an equivalent sequential abstract state machine B.” Blass and Gurevich extend it to parallel algorithms [3]. Dershowitz and Gurevich add a fourth axiom, that initial operations are computable, and from it derive Church’s thesis [4].

3.2 Definitions

Definition 1 (the Form). A program is a tuple

$$P = (S, E, \delta, D, \lambda)$$

- S, the state space: structures over a fixed vocabulary Σ. Each state is a database instance in Codd’s sense [5].
- E, the event alphabet, including the logical clock tick.
- $\delta : S \times E \to S$, the transition: total, deterministic and pure, written as a finite set of guarded updates “if φ then $f(\bar{t}) := t_0$”, each step reading a bounded set of locations.
- $D = (s_0, L)$, the durable store: an initial state and an append-only event log $L = e_1, \ldots, e_n$.
- $\lambda : S \times E \to O$, the output map. A Mealy machine reads both arguments [6]; a Moore machine reads the state only [7].

Definition 2 (Form and fit). The *form* is the tuple shape with four invariants: δ is deterministic and pure; exploration is bounded; the vocabulary is fixed; state is the fold of the log,

$$\mathrm{state}(D) = \mathrm{foldl}(\delta, s_0, L).$$

The *fit* is the data that fills the shape: the vocabulary Σ, the alphabets E and O, the rule program Π that defines δ, the output rules that define λ, and the initial data s_0. In Gurevich’s terms the fit is the vocabulary, the ASM program and the initial structures. In Codd’s terms Σ is the schema and s_0 is the instance. In product-line terms a fit is a feature selection [8]. In syntax-guided synthesis (SyGuS) terms a fit is a term from a grammar that meets a specification [9].

Definition 3 (fit equivalence). Two fits are equivalent when, for every log L, their output sequences under λ are equal.

Definition 4 (fit language). A grammar whose terms denote fits. Programming languages, SCXML documents, XState configurations and `.rce` specifications are fit languages. A fit language is complete when every ASM program over Σ has a term in it.

Definition 5 (control projection). Split $\Sigma = \Sigma_c \cup \Sigma_d$ into finite control symbols and data relations. The statechart of P is the projection of δ onto Σ_c: states are values of Σ_c, transitions carry an event and a guard over Σ_d, and actions are updates to Σ_d. Harel’s statecharts give the hierarchy and the orthogonal regions [10].

Definition 6 (fit e-graph). For a fit F and a set R of sound rewrites, the fit e-graph $EG(F, R)$ is the saturated e-graph of F's rule terms under R [11, 12]. Each e-class holds fits equivalent by Definition 3.

Definition 7 (joule cost). $\mathrm{cost}_J(t)$ is the estimated energy (`est_j`) to run term t once on a named substrate, weighted by the expected number of runs.

3.3 Why "database functional state machine"

A finite relational structure is a database instance, so a Gurevich state is a database and each step is a bounded transaction. *Database*: S is a relational structure. *Functional*: δ and λ are pure. *State machine*: runs are folds of δ over events. Moseley and Marks draw the same line in architecture [13]. Event sourcing draws it in application state: "Capture all changes to an application state as a sequence of events" [14]. Dedalus draws it in logic, where mutable state is a relation indexed by time [15].

3.4 Propositions

Proposition 1 (the Form captures every sequential algorithm). For every sequential algorithm A there is a Form instance P whose runs are the runs of A.

Proof sketch. Theorem 6.13 gives an ASM B with the states, initial states and one-step map of A. Set $E = \{\varepsilon\}$, $\delta(s, \varepsilon) = \tau_B(s)$, $D = (s_0, \varepsilon^n)$ and λ the identity. The fold after n ticks is $\tau_A^n(s_0)$. By Gurevich's Corollary 3.3, P and A have the same runs. ∎

Proposition 1b (interactive programs). Extend Σ with a nullary input symbol set to e before each step; then $\delta(s, e) = \tau(s[in := e])$ is a sequential algorithm over the extended vocabulary, and Proposition 1 applies. The construction is ours.

Proposition 1c (computability). Under the Dershowitz and Gurevich axiom, every Form instance computes a Turing-computable function [4, 16, 17]. The Form adds no power. It gives every program in the class one shape.

Proposition 1d (proofs are fits). "For each proof of a given proposition, there is a program of the corresponding type" [18]. A type is a specification and a proof is a fit that meets it. The correctness gate is the existence of that proof or of a passing acceptance test.

Proposition 2 (one generator). One function $G : \mathrm{Fit} \to \mathrm{Program}$ generates a runnable program for every fit and reads no domain. *Basis:* every program is a Form instance, so G emits a typed store, an event type, compiled rules, a log with a fold and an output dispatcher. Scaffolders such as Rails generators, JHipster and Yeoman are G restricted to small fit languages. Product-line engineering starts from "the systematic discovery and exploitation of commonality across related software systems" [8]; the commonality is the Form.

Proposition 3 (Mixture of Limits as extraction). Extraction with cost_J returns the least-joule term the extractor finds in the e-class of F. *Proof sketch.* The rewrites are sound, so every term in the e-class has F's observable behavior and passes the same gate. Extraction minimizes the cost, and the cost is joules. ∎ This is Mixture of Limits applied to code: keep the minimal fit that does the task, and the rest is waste.

Proposition 4 (replay determinism). If δ reads no wall clock, orders updates without names and logs every input, two replays of one D give identical state hashes at every step.

Proposition 5 (statechart projection is sound). Project each state of a run onto Σ_c. The result is a run of the emitted statechart under the same events.

Proposition 6 (coordination budget). Rules tagged monotone run on replicas without coordination; only non-monotone rules need a coordination point. *Basis:* the CALM theorem, "the programs that have consistent, coordination-free distributed implementations are exactly the programs that can be expressed in monotonic logic" [19].

Proposition 7 (intelligence as a joule derivative). Let X count tasks through the gate and J the joules spent. $\iota = dX/dJ$ is read in hindsight from D, which records every event and its `est_j`. Extraction under cost_J holds the gate fixed and does not raise J, so at equal X it does not lower ι (Agency, Section 2.3).

3.5 Graph, e-graph and statechart

Structure	Part of the Form	What it decides
Statechart	Guards of δ over Σ_c	Which transition fires
Dataflow graph	Read and write dependencies of δ and λ (Kahn process networks [20])	What runs, and in what order
E-graph	The equivalence class of the fit	Which equivalent fit runs

The statechart and the graph describe one fit. The e-graph describes every fit equivalent to it. Extraction chooses among them by joules.

---

4. The engine: gears, gate and receipts

4.1 The engine is a Form

RCE runs software as a graph of small deterministic handlers. A handler that cannot decide raises an escalation. A resolver answers the escalation once, and the answer is installed as a permanent handler, so the next case of the same shape runs with no resolver. The engine meets every Form invariant, each checked by a named test in `tests/form_laws.rs`:

Invariant	What the engine does
Reload is a verified fold	Reload reads the log verbatim, recomputes every hash and link, and folds the rest with δ. Nothing is re-stamped. A broken chain refuses to load.
Name-free tick order	Handlers run in topological rank order, then by a name-free node key. Outputs are staged and delivered after all handlers run, as in ASM semantics. Renaming every node gives the same anonymous state hash at every entry.

Invariant	What the engine does
No silent drop	A full mailbox holds the message in an ordered queue and logs `Backpressure`. With capacity 1 and 10 tasks, 9 `Backpressure` entries are logged and all 10 tasks close.
Payloads are in the log	Inputs, ticks, external handler results, installs and faults are log entries. The state hash after every entry matches a fold from an empty engine, on 1,000 random logs.
Logical clock	Liveness counts ticks, not wall time.
Stable hashes	SHA-256 (Secure Hash Algorithm, 256-bit) over canonical JSON, keys sorted at every depth, for entries, states, shapes and rule ids.

4.2 The gears

Each gear is tried only when the gear below it does not pass the gate. That order is one fixed cascade, and the fixed cascade is one Mixture of Limits instance.

Gear	Method	Lineage
G0 Lookup	A shared handler library keyed by SHA-256 shape hash; for code repair, localization to ranked functions	Code search and retrieval [21]
G1 Rules	Deterministic, guarded repair templates and schema-derived validators, each checked for mutation sensitivity	PAR, TBar, AVATAR, FixMiner [22, 23, 24, 25]; codemods such as Coccinelle [26]
G2 Synthesis	Enumeration over the handler grammar, solvers and e-graphs	Sketching, SyGuS, egg [27, 9, 12]
G3 to G5 Models	A typed decision model, a local small model, then a remote model, last	Model cascades [28]

4.3 The gate

A candidate is promoted only when it passes the gate. The gate replays the node's recorded inputs, runs the candidate in shadow on fresh samples with a coverage bound (the rule of three at a declared error budget), checks the output schema, and requires the proof certificate of Section 5. A rejected candidate is logged with its counterexample.

4.4 Receipts

Every promotion, refusal, deferral and install is a mutation in an append-only log chained with SHA-256, so tampering with any past entry breaks the chain [29, 30]. Each handler's cost is priced from interpreter operation counts with the host cost table of FGIR, the native-code backend of flowG, and labeled `est_j`. Each foreman call carries its receipt. Track B adds a package energy counter, labeled `reported_j`.

4.5 The Form compiler

`rce form` takes a specification and its fit and emits five targets from one Form IR:

```
.rce spec + fit
  lower ─► normalize ─► egg e-graph, equality saturation (+ admitted proposals)
  ─► extract (cost = est_j) ─► Form IR (typed relations, regions, rules with SHA-256 ids)
     ├─ Rust crate (pure step, typed state, SHA-256 log and fold, tests)
     │  └─ WASM (the same crate on wasm32-wasip1)
     ├─ Python (standard library only)
     ├─ XState v5 machine
     └─ SCXML document, executed by SCION
```

The IR fixes one behavior at every numeric edge: checked 64-bit integer overflow is an error and never wraps; integer `/` truncates toward zero ($-7/2 = -3$); `%` takes the dividend's sign ($-3 \bmod 2 = -1$); division by zero is an error; floats are IEEE 754 binary64 and a non-finite result is an error; null and absent are distinct; time is the logical clock only. Every target is generated. None is authored.

A process, HTTP or closure handler has no term, so its logged results compile to a recorded oracle keyed by the SHA-256 of the canonical input. `rce scan DIR --form` lifts existing Rust, Python, TypeScript and Go into the Form's schema, and every lifted element is labeled as a syntactic lift.

---

5. Verification

5.1 Five targets, one trace

`tests/form_targets.rs` drives an eleven-operator arithmetic specification with 29 edge events plus 500 seeded random edge values. The generated Rust, Python, WASM, XState and SCXML targets print the reference trace byte for byte, including the arithmetic edges.

5.2 Rewrites proved in Z3

Every built-in e-graph rewrite is proved in the Z3 SMT solver (satisfiability modulo theories) [31] against a quantifier-free encoding of the Form's term evaluation and arithmetic: values as a datatype, 64-bit bit-vectors with checked overflow, signed division and remainder with the dividend's sign, and binary64 floats. A rewrite that changes overflow, escalation order or a message gets no proof. A proposed equivalence enters the e-graph only with a proof.

5.3 Kani proofs of the kernel

The engine's kernel (hash chain, fold, name-free positions and handler evaluation) is checked with the Kani model checker for Rust [32]. Handler evaluation is the fixed point of a non-recursive step, and Kani proves that step total and deterministic per constructor, by structural

induction. Ten harnesses are verified, with 0 of 505 checks failed:

Harness	Property
`chain_appended_verifies`	An appended hash chain verifies
`chain_detects_tampering`	A changed entry breaks the chain
`fold_equals_replay`	The fold equals replay
`fold_rejects_wrong_state`	A fold that disagrees with the stored state is refused
`position_is_name_independent`	Rule positions do not depend on node names
`reason_tokens_are_name_independent`	Reasons are name-free by whole tokens
`handler_core_leaves`	Leaf handlers are total and deterministic
`handler_core_validate`	Validation is total and deterministic
`handler_core_pipeline`	Pipelines are total and deterministic
`handler_core_ifhas`	Guards are total and deterministic

5.4 Promotion gate on the proof certificate

The Kani run writes a certificate whose hash is stored in `proofs/kani.cert`. The engine, the log, rule ids and handler evaluation route through the proved kernel, and promotion requires the certificate.

5.5 The model as a proposer behind the check

A model may propose equivalences. It may not admit them. With a local 1.5-billion-parameter coding model (Qwen2.5-Coder-1.5B under llama.cpp) on the demonstration specification, the proposer offered 16 equivalences. The check admitted 3, by randomized trials and a Z3 proof, and rejected 13. A deterministic proposer of 87 familiar identities had 9 admitted, all structural, and every arithmetic identity rejected for fault order, non-numbers or overflow.

5.6 Brownfield agreement

The lift reads the same order machine from Rust, Python, TypeScript and Go fixtures, runs the original code on 40 seeded sequences of 30 calls, and agrees on all 1,200 calls per language. A frozen benchmark, preregistered before any result, re-expressed three existing sales-outreach programs as Forms. The Forms agree with the original code on 4,500 of 4,500 recorded inputs, and extraction lowers `est_j` on each (for example 1.69×10^{-8} J to 1.49×10^{-8} J).

---

6. Results

Every run below has a locked preregistration whose SHA-256 is in the repository, and every artifact is kept.

6.1 Track A: a frozen repair corpus

Corpus20 holds 20 real regressions from five open-source libraries (strsim, heck and shlex in Rust; packaging and idna in Python). Each task has a failing test and a gold patch, and tests run sandboxed with no network. A task is valid when the gold patch solves it and no patch does not. Gate: the task's tests pass.

Run	Valid tasks	Solved	Model calls	Calls per solved task	By gear
corpus20, earlier solver	20	10 of 20	31	3.10	G1 rules 9, local model 1
corpus20-2026-10-11b	20	**20 of 20**	**0**	**0.00**	G1 rules 20

In corpus20-2026-10-11b, 18 of 20 solver fixes leave the edited file byte-identical to gold. The other 2 are equivalent: idna-label-length writes `>= 64` for `> 63`, equal on integers, and packaging-dev-of-post puts the added conjunct first. All three preregistered hypotheses held: every task is valid; the solver resolves 20 of 20 with a patch equal to gold; no task reaches the model.

6.2 Track B: joules per solved task on jetson-hub

Track B runs the same corpus on jetson-hub, a Linux laptop with a 14-core x86 processor. The benchmark and the model server ran pinned to CPUs 2 to 5. The meter is RAPL (Running Average Power Limit), the processor's on-chip package energy counter. **Baseline method:** before each run the meter samples the package for 10 s with none of our processes running and takes that as idle power; `reported_j` is the run's counter delta minus idle power times the run's duration. The machine was busy with CI (continuous integration) runners throughout, and the idle sample moved between 25.7 W and 57.8 W. Every figure is `reported_j`. `measured_j` is reserved for a wall meter.

Solver	Solved	Model calls	`reported_j` per solved task	Seconds
RCE solver, earlier (2026-10-10)	11 of 20	29	712.4	695
RCE solver, new (2026-10-11b)	**20 of 20**	**0**	**84.2**	154
Gold patch	20 of 20	0	2.0	5

The new solver passes the gate on every task, at 8.5 times fewer reported joules per solved task than the earlier solver. Gold, the cost of applying a known fix and running the tests, is the floor of this corpus: 2.0 `reported_j` per task. The distance from 84.2 to 2.0 is the search that retrieval and proof still pay for.

6.3 The soaks converge

Track C runs the five example specifications under two resolvers for 20 rounds each and reports `est_j` per closed task. In the 2026-10-09 run, every specification reaches a round with zero new escalations, in round 4 to 7 for both resolvers, and from that round the all-in `est_j` per closed

task equals the handler floor: about 2 to 5 nanojoules per task. The gear stack resolved every case from G0 and G1, with zero model calls. In the 2026-10-10 live2 run, on live traffic only, all 12 runs reach a quiet round before round 20 (web-api 4, joule-db-core 7, follow-bible-ingest 6, follow-bible-ingest-schemas 8, ask-server 5, soak-g2-schemas 5), and every run closes tasks through the gate. `measured_j` is empty in every row.

---

7. Reuse evidence

The second path rests on an engineering fact: code is reused at every scale. Each method below measures it differently.

Method	Source	Finding
File-level clone mapping	DéjàVu, Lopes et al. 2017 [33]	Of 428 million files in 4.5 million non-fork GitHub projects, 85 million are unique: "70% of the code on GitHub consists of clones of previously created files." In JavaScript, 6% of files are distinct.
Line-level redundancy	Gabel and Su 2010 [34]	Against 420 million lines, "between 50% and 100% of each project is redundant" at about one line.
Statistical language models	Hindle et al. 2012 [35]	Code is "very repetitive, and in fact even more so than natural languages."
Corpus near-deduplication	The Stack [36]	Near-deduplication removes 54.7% of The Stack v1 (6.4 TB to 2.9 TB) and 52.4% of v2 (67.5 TB to 32.1 TB).
Whole-archive preservation	Software Heritage	About 30 billion source files from 445 million origins, each stored once by the hash of its bytes.
Mined fix patterns	PAR, TBar [22, 23]	PAR found "several common fix patterns" in more than 60,000 human patches. TBar's 15 pattern categories fix 43 Defects4J bugs correctly in the standard pipeline, and 74 of 101 with perfect fault localization, with no learning.
Cross-language tasks	Rosetta Code	1,360 tasks, each one concept implemented across many of 1,009 known languages.

Each row is an input to the engine. Clones and dedup say a fit usually exists, which is G0. Recurring fixes say adaptation is mostly a small catalog of guarded rewrites, which is G1. Line-level redundancy shrinks the search space of G2. Track A is this evidence at small scale: every fix came from G1 rules, and 18 of 20 match the human fix byte for byte.

---

8. Measurement protocols for what comes next

Each protocol is fixed before its run: preregistration committed, seeds and solver commit recorded, every artifact kept. Gate first, joules second, labels always.

8.1 Defects4J

Defects4J holds 854 active real Java bugs, each with reproducing tests [37].

1. Run a 100-bug subset first, offline, through `defects4j checkout`.
2. Run G0, then G1, then G2, then a local model last. Record the gear that closed each bug.
3. Gate: the triggering tests and the full suite pass. Report plausible and correct counts separately; correct means equivalent to the developer patch on review.
4. Report model calls per correct fix and joules per correct fix, labeled.
5. Reference points: TBar's 43 correct fixes and its 74 of 101 under perfect localization.
6. Contamination check: flag every bug whose fix appears verbatim in G0's index, and report with and without flagged bugs.

8.2 SWE-bench Lite, offline

SWE-bench Lite is a 300-task subset of real GitHub issues resolved by hidden tests [38].

1. Use the official Docker harness with no network.
2. First run: G0 and G1 only, zero model calls. This is the floor of solving without a model.
3. Second run: add a local model as the last gear.
4. Report the resolve rate, model calls per resolved task and joules per resolved task, labeled.
5. Contamination check as in 8.1. SWE-Bench+ found that 32.67% of successful patches had the solution in the issue text [39].

8.3 A wall meter

1. Attach a wall meter or a shunt meter on the host's supply, logged at a stated rate with its clock synchronized to the run log.
2. Idle machine: CI runners and other builds stopped, and a quiet window gate checked before each block and recorded.
3. Subtract a measured idle baseline, and state the method.
4. Fill `measured_j` from the meter alone. Report `reported_j` (RAPL) and `est_j` beside it in separate columns. Never sum across labels.
5. Repeat Track B on corpus20, then on the Defects4J subset.

Proposition 8 (floor approach). As G0 and G1 coverage grows, joules per solved task approach the gold floor of applying a known fix and running the tests. Tracks A and B on the Defects4J subset under the wall meter test it.

---

9. Ties to the companion papers

Mixture of Limits. Metered Commit Gates, Part II states the philosophy: a minimal amount of anything is needed to accomplish a task, and any more is waste. RCE applies it three times. The gear order runs the cheapest sufficient gear first. Promotion pays a resolver once per new case,

after which every repeat call would be waste. Extraction keeps the least-joule fit among equivalent fits (Proposition 3).

Metered Commit Gates. Part I says a proposal may not authorize itself. In RCE a model proposes and the gate commits: a candidate enters only with tests passed, a coverage bound met and a proof certificate, and every decision is a receipt in the hash chain.

The Universal Law of Agency. The agency paper defines intelligence as the derivative of agency in joules, $\iota = dX/dJ$, read in hindsight. The RCE log is that hindsight record: X is tasks through the gate and J is their joules, labeled. A deferral with a receipt is the refusal clause of the gate.

Task Satiation. Task Satiation vs. Aggregate Rebound in AI Demand says that once a task's completion predicate is true, more synthesis on that task buys nothing. RCE is per-task satiation inside one system: once a handler covers a case, further inference on that case is zero. The aggregate demand question belongs to that paper and to the economics companion.

References

[1] C. E. Shannon, "Prediction and Entropy of Printed English," *Bell System Technical Journal* 30(1), 1951. doi:10.1002/j.1538-7305.1951.tb01366.x

[2] Y. Gurevich, "Sequential Abstract State Machines Capture Sequential Algorithms," *ACM Transactions on Computational Logic* 1(1), 2000. doi:10.1145/343369.343384

[3] A. Blass and Y. Gurevich, "Abstract State Machines Capture Parallel Algorithms," *ACM Transactions on Computational Logic*, 2003. doi:10.1145/937555.937561

[4] N. Dershowitz and Y. Gurevich, "A Natural Axiomatization of Computability and Proof of Church's Thesis," *Bulletin of Symbolic Logic*, 2008. doi:10.2178/bsl/1231081370

[5] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, 1970. doi:10.1145/362384.362685

[6] G. H. Mealy, "A Method for Synthesizing Sequential Circuits," *Bell System Technical Journal*, 1955. doi:10.1002/j.1538-7305.1955.tb03788.x

[7] E. F. Moore, "Gedanken-Experiments on Sequential Machines," *Automata Studies*, 1956. doi:10.1515/9781400882618-006

[8] K. Kang et al., "Feature-Oriented Domain Analysis (FODA) Feasibility Study," CMU/SEI-90-TR-021, 1990.

[9] R. Alur et al., "Syntax-Guided Synthesis," FMCAD 2013. doi:10.1109/FMCAD.2013.6679385

[10] D. Harel, "Statecharts: A Visual Formalism for Complex Systems," *Science of Computer Programming*, 1987. doi:10.1016/0167-6423(87)90035-9

[11] R. Tate, M. Stepp, Z. Tatlock and S. Lerner, “Equality Saturation,” POPL 2009. doi:10.1145/1480881.1480915

[12] M. Willsey et al., “egg: Fast and Extensible Equality Saturation,” POPL 2021. doi:10.1145/3434304

[13] B. Moseley and P. Marks, “Out of the Tar Pit,” 2006.

[14] M. Fowler, “Event Sourcing,” 2005. https://martinfowler.com/eaaDev/EventSourcing.html

[15] P. Alvaro et al., “Dedalus: Datalog in Time and Space,” LNCS 2011. doi:10.1007/978-3-642-24206-9_16

[16] A. M. Turing, “On Computable Numbers,” *Proceedings of the London Mathematical Society*, 1937. doi:10.1112/plms/s2-42.1.230

[17] A. Church, “An Unsolvable Problem of Elementary Number Theory,” *American Journal of Mathematics*, 1936. doi:10.2307/2371045

[18] P. Wadler, “Propositions as Types,” *Communications of the ACM*, 2015. doi:10.1145/2699407

[19] J. M. Hellerstein and P. Alvaro, “Keeping CALM,” *Communications of the ACM*, 2020. doi:10.1145/3369736

[20] G. Kahn, “The Semantics of a Simple Language for Parallel Programming,” IFIP 1974, pp. 471 to 475.

[21] H. Husain et al., “CodeSearchNet Challenge,” 2019. arXiv:1909.09436

[22] D. Kim, J. Nam, J. Song and S. Kim, “Automatic Patch Generation Learned from Human-Written Patches” (PAR), ICSE 2013. doi:10.1109/ICSE.2013.6606626

[23] K. Liu et al., “TBar: Revisiting Template-based Automated Program Repair,” ISSTA 2019. arXiv:1903.08409

[24] K. Liu et al., “AVATAR: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations,” SANER 2019. arXiv:1812.07270

[25] A. Koyuncu et al., “FixMiner: Mining Relevant Fix Patterns for Automated Program Repair,” 2018. arXiv:1810.01791

[26] Y. Padioleau et al., “Documenting and Automating Collateral Evolutions in Linux Device Drivers” (Coccinelle), EuroSys 2008. doi:10.1145/1352592.1352618

[27] A. Solar-Lezama et al., “Combinatorial Sketching for Finite Programs,” ASPLOS 2006. doi:10.1145/1168857.1168907

[28] L. Chen, M. Zaharia and J. Zou, “FrugalGPT,” 2023. arXiv:2305.05176

[29] S. Haber and W. S. Stornetta, “How to Time-Stamp a Digital Document,” *Journal of Cryptology*, 1991. doi:10.1007/BF00196791

[30] S. A. Crosby and D. S. Wallach, “Efficient Data Structures for Tamper-Evident Logging,” USENIX Security 2009.

[31] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS 2008. doi:10.1007/978-3-540-78800-3_24

[32] Kani Rust Verifier. https://github.com/model-checking/kani

[33] C. V. Lopes et al., “DéjàVu: A Map of Code Duplicates on GitHub,” OOPSLA 2017. doi:10.1145/3133908

[34] M. Gabel and Z. Su, “A Study of the Uniqueness of Source Code,” FSE 2010. doi:10.1145/1882291.1882315

[35] A. Hindle et al., “On the Naturalness of Software,” ICSE 2012. doi:10.1109/ICSE.2012.6227135

[36] D. Kocetkov et al., “The Stack: 3 TB of Permissively Licensed Source Code,” 2022. arXiv:2211.15533; The Stack v2 dataset card, https://huggingface.co/datasets/bigcode/the-stack-v2

[37] R. Just, D. Jalali and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” ISSTA 2014. doi:10.1145/2610384.2628055

[38] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. arXiv:2310.06770

[39] R. Aleithan et al., “SWE-Bench+: Enhanced Coding Benchmark for LLMs,” 2024. arXiv:2410.06992